

Lecture-10, 11, and 12

The Binary Search Trees

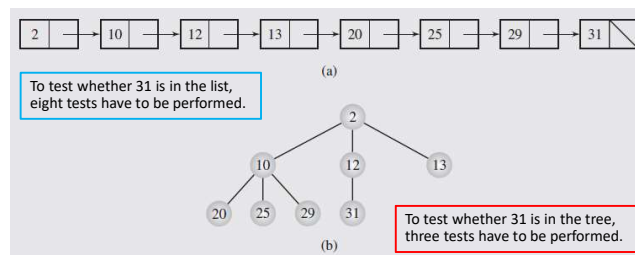
Part-I

Linear vs Hierarchical Data Structures

- Data structures like *arrays*, *stacks*, *queues*, and *linked lists* organize elements in a sequential order, defining relationships only in terms of “before” or “after.”
- These structures are efficient for specific operations but cannot represent relationships beyond a single dimension, such as hierarchical relationships like “below” or “above.”
- To represent hierarchical relationships, such as parent-child or organizational structures, we use tree data structures.
- A file system is an excellent example where a tree structure outperforms linear structures in representing directory hierarchies.

Introducing Trees: A New Data Type

- Trees are hierarchical structures composed of nodes and edges, designed to represent relationships that go beyond linear sequences.
- They start with a root node and branch out into multiple levels.
- Unlike natural trees, computational trees are inverted, with the root at the top and leaves at the bottom.
- Trees are ideal for applications like representing file systems, decision-making processes, and organizational charts.



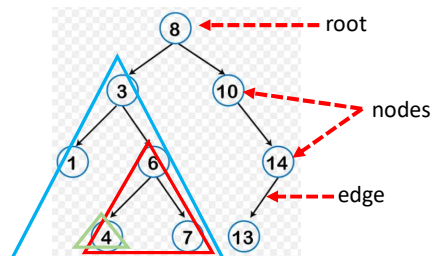
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

3

Understanding Tree Structure

- A tree consists of nodes connected by edges, forming a hierarchy.
- The root node is the starting point, and every other node is connected to it via a unique path.
- Each subtree rooted at a node can be considered an independent tree, enabling modularity and recursive design.
- This structure allows trees to model complex relationships efficiently.



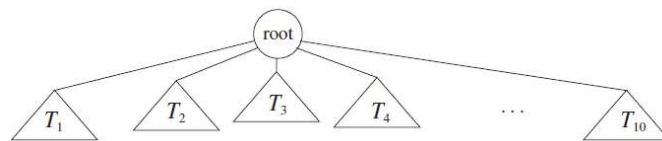
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

4

Defining Trees Recursively

- A tree is defined recursively as:
 1. An empty structure is an empty tree.
 2. If $t_1, t_2, \dots, t_k$ are disjoint trees, then a root node connected to these trees forms a tree.
- Recursive definitions make trees straightforward to implement in programming, supporting operations like traversal and modification.



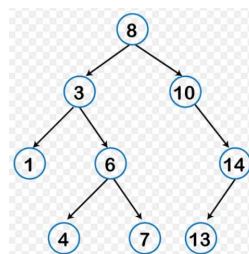
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

5

Basic Properties of Trees

- Trees have nodes connected by edges, with no cycles.
- Each node has exactly one parent (except the root) and may have multiple children.
- Internal nodes contribute to the tree's structure, while leaves (terminal nodes) represent endpoints.
- These properties make trees a subset of graphs but distinct due to their hierarchical nature.



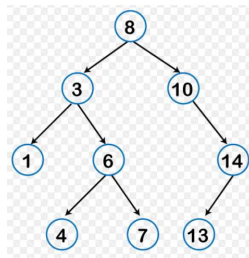
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

6

Paths, Nodes, and Root in a Tree

- A path is a sequence of nodes where consecutive nodes are connected by edges.
- The root is the unique node without a parent, serving as the tree's origin.
- Leaves are nodes without children, and internal nodes connect the root to the leaves.
- Paths and nodes define how elements in the tree are related and accessed.



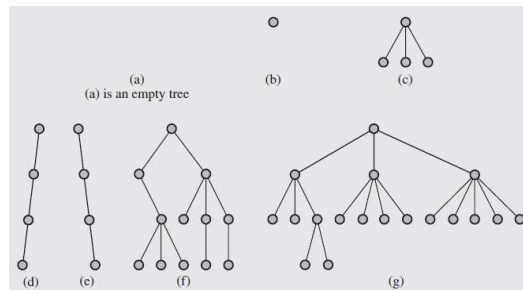
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

7

Tree vs Graph: Key Differences

- Trees have no cycles, while graphs can.
- In a tree, there is exactly one unique path between any two nodes, ensuring no ambiguity.
- Graphs may allow multiple paths and disconnected components.
- For example, a network diagram is a graph, but a family tree is a tree



Example trees

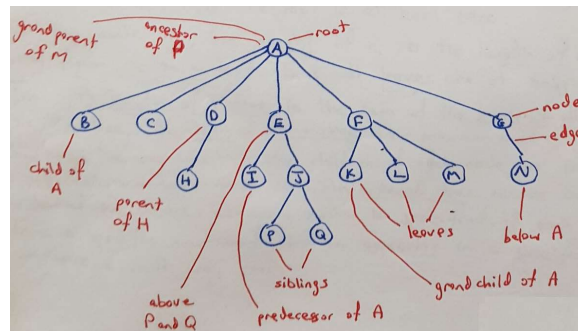
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

8

Hierarchy in Trees: Parent, Child, and Leaves

- Each node (except the root) has one parent and may have multiple children.
- Leaves are endpoints, while internal nodes serve as connections within the tree.
- These relationships are fundamental to operations like traversals, searches, and modifications.
- In applications, such as binary search trees, these relationships define the structure and utility of the tree.



Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

9

Internal and External Nodes

- Internal nodes have at least one child and form the backbone of the tree.
- External nodes (leaves) have no children and represent endpoints.
- Every node in the tree is part of a subtree, reinforcing the recursive nature of trees.
- For example, in a directory tree, folders are internal nodes, and files are external nodes.

Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

10

Tree Terminology: Depth, Height, and Path Length

- **Path length:** The number of edges in a path.
- **Level:** The number of edges from the root to a node (root is at level 0).
- **Height:** The longest path from a node to a leaf (leaves have height 0).
- **Depth:** The maximum level of any node in the tree.

These metrics help in analyzing tree structures and their efficiency in operations.

Ordered vs Unordered Trees

- Ordered trees specify the sequence of children for every node, important for certain applications.
- Unordered trees do not impose any sequence, allowing for more flexibility.
- Multiway trees are ordered trees where nodes can have a fixed number of children in a specific order.
- Ordered trees are used in decision-making processes, while unordered trees are common in social networks.

Multiway Trees and Binary Trees

- Multiway trees allow nodes to have more than two children, often used in database indexing and file systems.
- Binary trees limit each node to at most two children, simplifying their structure and operations.
- In binary trees, the two children are explicitly designated as the left and right child.
- Examples: Binary search trees are used for efficient searching, while B-trees are used in databases.

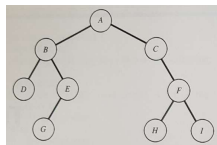
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

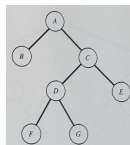
13

Properties of Binary Trees

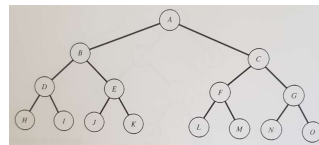
- Binary trees restrict nodes to zero, one, or two children, enhancing simplicity and efficiency.
- Common types include:
 - **Binary tree**: A tree where each node has at most two children.
 - **Strictly binary tree**: A tree where each internal node has exactly two children.
 - **Full binary tree**: A tree where all nodes, except leaves, have exactly two children.
- Binary trees support algorithms like sorting, searching, and data compression.



Binary tree



Strictly
binary tree



Full binary tree

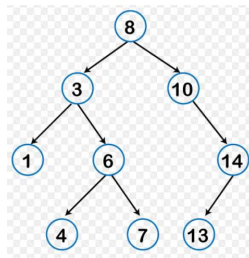
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

14

Visualizing Trees and Binary Trees

- Trees are typically visualized with the root at the top and edges pointing downward.
- Binary trees explicitly label children as left or right to maintain order.
- Visualization aids in understanding relationships and designing tree-based algorithms like traversals and balancing.



Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

15

Applications of Binary Trees

- Binary trees are widely used beyond searching and organizing data.
- **Key Application:** Compiler design.
 - Binary trees are utilized to represent and evaluate expressions.
- **Expression Trees:**
 - Leaves: Represent operands (constants or variable names).
 - Internal nodes: Contain operators (e.g., +, -, *, /).
 - Structure:
 - Typically binary when operators are binary (e.g., addition, multiplication).
 - Can include nodes with one child (e.g., unary operators like -).
 - Nodes with more than two children are possible for specific cases.

Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

16

Expression Trees: An Example

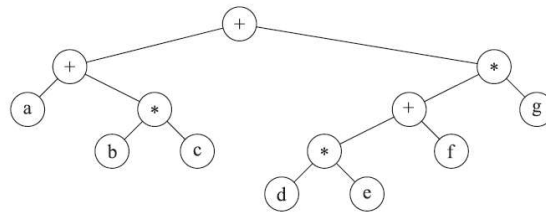


Figure Expression tree for $(a + b * c) + ((d * e + f) * g)$

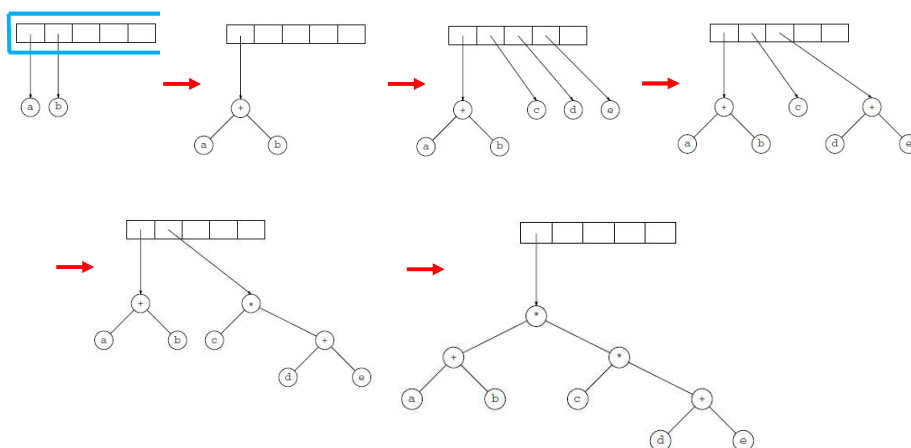
Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

17

Constructing an Expression Tree (with Stack)

- Example: $a b + c d e + * *$



Lecture-10

COMP217 by Dr. Ferhun Yorgancıoğlu

18

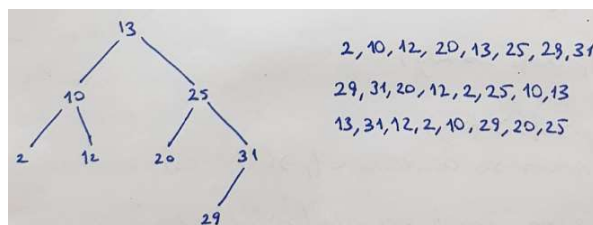
Lecture-10, 11, and 12

The Binary Search Trees

Part-II

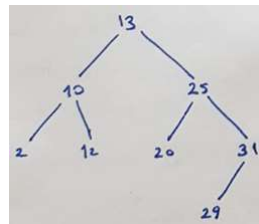
Tree Traversals Overview

- What is Tree Traversal?
 - Tree traversal is the process of visiting each node in the tree exactly once.
 - It may be interpreted as linearizing the tree by putting all nodes on one line.
 - Traversal ensures that each node is visited exactly once, but it doesn't define the order of visits.
 - As a result, for a tree with n nodes, there are $n!$ possible traversal orders.
- Types of Traversals
 - Breadth-First Traversal
 - Depth-First Traversal



Breadth-First Traversal

- What is Breadth-First Traversal?
 - Visit nodes level by level, starting from the lowest (or highest) level.
 - Visit nodes on each level from left-to-right (or right-to-left).
- Example: Top-Down, Left-to-Right
 - Visit nodes from top to bottom and left to right.



13, 10, 25, 2, 12, 20, 31, 29

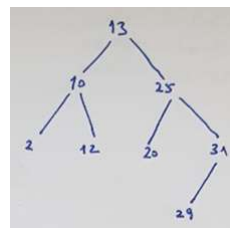
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

21

Breadth-First Traversal: Implementation

```
void breadth_first_traversal(p)
{
    q = empty_queue();
    if (p != NULL)
    {
        enqueue(q, p);
        while (!is_empty(q))
        {
            p = dequeue(q);
            visit(p);
            if (is_left(p))
                enqueue(q, p->left);
            if (is_right(p))
                enqueue(q, p->right);
        }
    }
}
```



Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

22

Depth-First Traversal

- What is Depth-First Traversal?
 - Proceed as far as possible down one branch, then backtrack and explore the next branch.
 - This definition doesn't specify when nodes are visited: before descending or after backtracking.
 - This traversal can be broken into three tasks:
 - V: Visiting the node
 - L: Visiting the left subtree
 - R: Visiting the right subtree

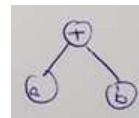
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

23

Depth-First Traversal: Ordering

- Possible Orders for Depth-First Traversal
 - There are six possible task orders:
 - VLR (Pre-order traversal)
 - VRL
 - LVR (In-order traversal)
 - LRV (Post-order traversal)
 - RLV
 - RVL
 - Reduced to Three Main Traversals:
 - VLR: **Pre-order** traversal (Node first) (+ a b)
 - LVR: **In-order** traversal (Left, Node, Right) (a + b)
 - LRV: **Post-order** traversal (Left, Right, Node) (a b +)

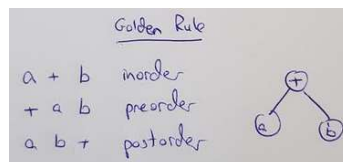
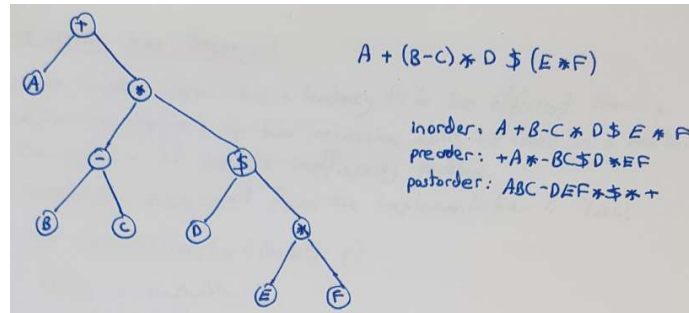


Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

24

Depth-First Traversals: In-order, Pre-order, Post-order



Lecture-11

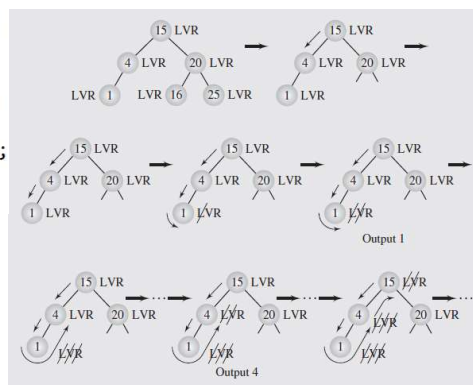
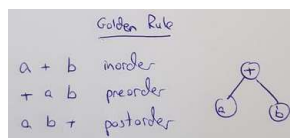
COMP217 by Dr. Ferhun Yorgancıoğlu

25

In-order Tree Traversal: Implementation

```

void inorder_traversal(NodePtr p) {
    if (p != NULL)
    {
        inorder_traversal(p->left);
        printf("%d ", p->info);
        inorder_traversal(p->right);
    }
}
    
```



Recursive function calls make it very simple to achieve the task.

Non-recursive implementation is also possible but difficult to write.

Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

26

Pre-order, Post-order Tree Traversal: Implementation

```
void preorder_traversal(NodePtr p) {  
    if (p != NULL)  
    {  
        printf("%d ", p->info);  
        preorder_traversal(p->left);  
        preorder_traversal(p->right);  
    }  
}
```

The order of recursive calls help us achieve the pre-/post-order traversals.

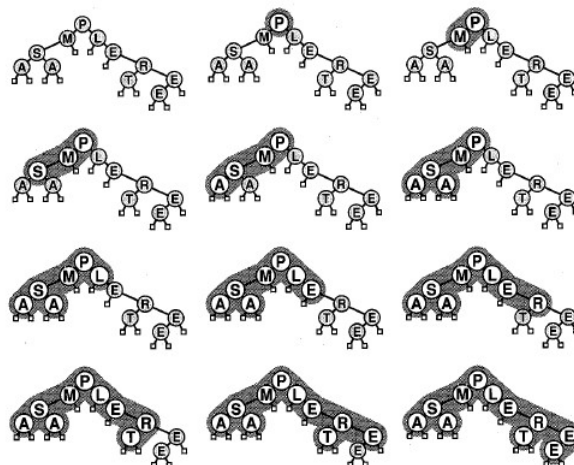
```
void postorder_traversal(NodePtr p) {  
    if (p != NULL)  
    {  
        postorder_traversal(p->left);  
        postorder_traversal(p->right);  
        printf("%d ", p->info);  
    }  
}
```

Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

27

Pre-order Traversal: Visualization

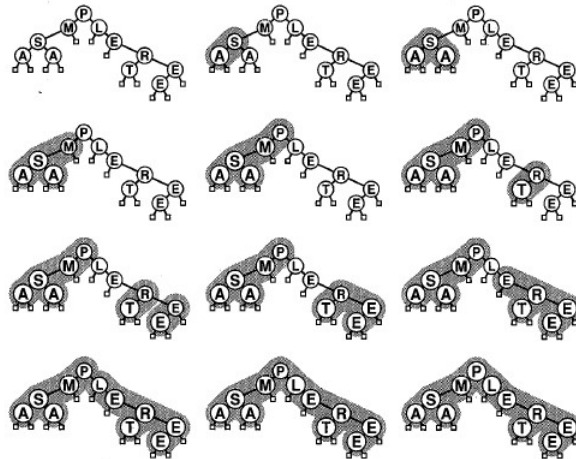


Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

28

Post-order Traversal: Visualization



Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

29

Searching a Tree: Process

- Compare the key: Check if the key to be found matches the value in the current node.
- Move Left or Right:
 - If the key is less than the node value, search the left subtree.
 - If the key is greater, search the right subtree.
- Stop the search:
 - If the key matches, the search is complete.
 - If there are no more nodes to visit, the key is not in the tree.
- Search Complexity:
 - Measured by comparisons:
 - The search complexity is determined by how many comparisons are needed to locate the key or determine its absence.

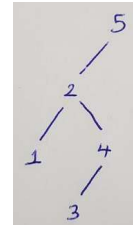
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

30

Searching a Tree: Implementation

```
NodePtr search_tree(NodePtr p, int key) {
    while (p != NULL)
    {
        if (p->info == key)
            return p;
        if (p->info > key)
            p = p->left;
        else
            p = p->right;
    }
    return NULL;
}
```



Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

31

Recursive vs Iterative Searching in Binary Trees

```
NodePtr search_iterative(NodePtr p, int key)
{
    while (p != NULL)
    {
        if (p->info == key)
            return p;
        if (p->info > key)
            p = p->left;
        else
            p = p->right;
    }
    return NULL;
}
```

```
int key = 3;
if (search_iterative(tree, key))
    printf("%d is found in the tree\n", key);
else
    printf("%d is NOT found in the tree\n", key);
```

```
NodePtr search_recursive(NodePtr p, int key)
{
    if (p != NULL)
    {
        if (p->info == key)
            return p;
        if (p->info > key)
            return search_recursive(p->left, key);
        else
            return search_recursive(p->right, key);
    }
    return NULL;
}
```



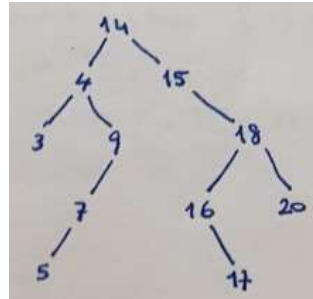
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

32

What is a Binary Search Tree (BST)?

- A binary tree is useful when decisions must be made at each point, with two options (left or right).
- Example Input:
 - 14, 15, 4, 9, 7, 18, 3, 5, 16, 4, 20, 17, 9, 14, 5



- **Properties of a Binary Search Tree (BST)**
 - For every node n:
 - All values in the left subtree are less than the value v stored in n.
 - All values in the right subtree are greater than v.
 - Avoids duplicate values:
 - Multiple copies of the same value are not stored in the tree.

Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

33

Inserting Nodes in a Binary Search Tree (BST)

Insertion vs Searching in a BST

- Searching a Binary Tree:
 - Searching does not modify the tree.
 - It scans the tree to find a key without altering its structure.
- Insertion in a Binary Tree:
 - Insertion involves modifying the tree by adding new nodes.
 - It requires navigating the tree to find the correct position based on the value to be inserted.
- Key Operations that Modify the Tree:
 - **Adding Nodes:** Insert new nodes to maintain the tree structure.
 - **Deleting Nodes:** Remove nodes and adjust the tree as necessary.
 - **Modifying Elements:** Update existing node values.
 - **Merging Trees:** Combine two or more trees into one.
 - **Balancing Trees:** Adjust the structure to reduce height and improve efficiency.

Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

34

Implementing the Binary Search Tree (BST) Structure

- A **node** in a binary tree typically contains:

- A key (value stored in the node).
- Left pointer (points to the left child).
- Right pointer (points to the right child).

```
struct node {  
    int info;  
    struct node* left, * right;  
};  
typedef struct node BST_Node;  
typedef struct node* BST_NodePtr;  
  
BST_NodePtr tree = NULL;
```

- Clearly, this is an application of the doubly-linked list structure.
- Additionally, note that the tree is represented by a pointer to the root node.

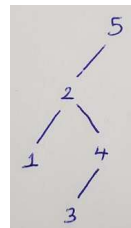
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

35

Algorithm for Insertion in a Binary Search Tree (BST)

```
int list[SIZE] = { 5, 2, 4, 1, 3 }, item;  
BST_NodePtr tree = NULL, p, q;  
tree = make_tree(list[0]);  
for (int i = 1; i < SIZE; i++)  
{  
    item = list[i];  
    p = q = tree;  
    while (item != p->info && q != NULL)  
    {  
        p = q;  
        if (item < q->info)  
            q = p->left;  
        else  
            q = p->right;  
    }  
    if (item == p->info)  
        printf("%d %s\n", item, " is a duplicate!");  
    else if (item < p->info)  
        set_left(p, item);  
    else  
        set_right(p, item);  
}
```



Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

36

Algorithm for Insertion in a BST – *cont.*

```
BST_NodePtr make_tree(int value)
{
    BST_NodePtr p = (BST_NodePtr)malloc(sizeof(BST_Node));
    p->info = value;
    p->left = NULL;
    p->right = NULL;
    return p;
}
void set_left(BST_NodePtr p, int value)
{
    if (p == NULL)
        printf("Void insertion!\n");
    else if (p->left != NULL)
        printf("Invalid insertion: nonempty left child!\n");
    else
        p->left = make_tree(value);
}
void set_right(BST_NodePtr p, int value)
{
    if (p == NULL)
        printf("Void insertion!\n");
    else if (p->right != NULL)
        printf("Invalid insertion: nonempty right child!\n");
    else
        p->right = make_tree(value);
}
```

Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

37

Eliminating Recursion in Tree Traversal

- Recursive vs Non-recursive Implementations
 - Recursion Efficiency:
 - Recursive implementations can be less efficient, especially when multiple recursive calls are involved, potentially doubling the inefficiency.
 - Can Recursion Be Eliminated?
 - **Yes!** Non-recursive methods can be used to achieve the same functionality more efficiently.

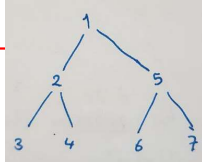
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

38

Non-recursive Implementation of Pre-order Tree Traversal

```
void pre_order(NodePtr p) {
    if (p != NULL) {
        printf("%3d", p->info);
        pre_order(p->left);
        pre_order(p->right);
    }
    return;
}
```



```
void iterative_preorder(NodePtr p) {
    Stack s = empty_stack();
    if (p != NULL) {
        push(s, p);
        while (!is_empty(s)) {
            p = pop(s);
            printf("%d", p->info);
            if (is_right(p))
                push(s, p->right);
            if (is_left(p))
                push(s, p->left);
        }
    }
}
```

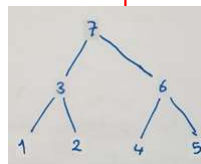
Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

39

Non-recursive Implementation of Post-order Tree Traversal

```
void iterative_postorder(NodePtr p) {
    NodePtr q = p;
    Stack s = empty_stack();
    while (p != NULL) {
        for (; p->left != NULL; p = p->left)
            push(s, p);
        while (p->right == NULL || p->right == q) {
            printf("%d", p->info);
            q = p;
            if (is_empty(s))
                return;
            p = pop(s);
        }
        push(s, p);
        p = p->right;
    }
}
```



Lecture-11

COMP217 by Dr. Ferhun Yorgancıoğlu

40

Lecture-10, 11, and 12

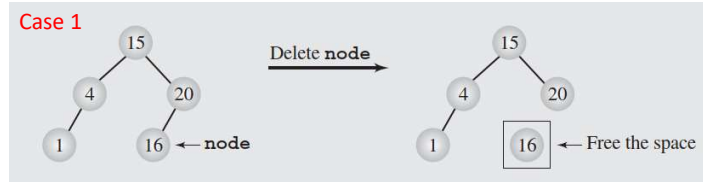
The Binary Search Trees

Part-III

Deleting Nodes from a Binary Search Tree

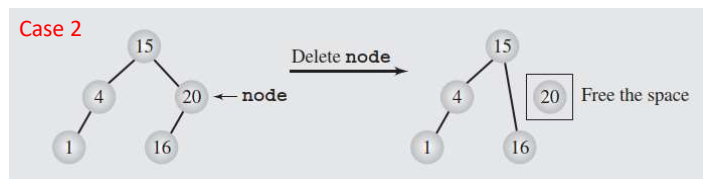
- Deleting a node in a binary search tree (BST) is necessary for maintaining its structure.
- The complexity of deletion depends on the node's position:
 - Leaf Node:
 - No children; simply remove the node by setting its parent's pointer to null.
 - One Child:
 - Update the parent's pointer to the node's child, lifting the child up a level.
 - Two Children:
 - More complex as the parent's pointer cannot directly connect to both children.
- This section covers two approaches for handling the two-children case:
 - deletion by **merging**, and
 - deletion by **copying**.

Deleting a Node: Leaf Case



Simply remove the node by setting its parent's pointer to null.

Deleting a Node: Single Child Case

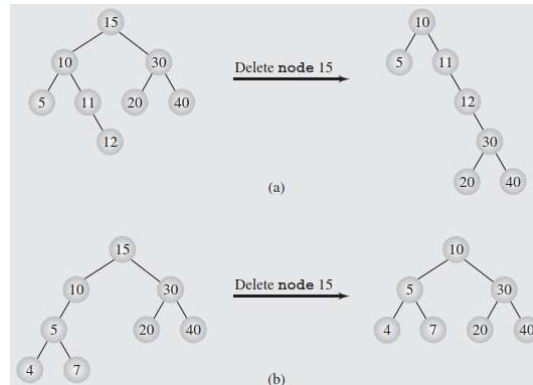


Update the parent's pointer to the node's child, lifting the child up a level.

Deletion by Merging

- This method combines the node's two subtrees into one and attaches it to the parent.
- In a BST, all keys in the right subtree are greater than those in the left. To merge, either:
 - Use the largest key in the left subtree as the parent of the right subtree, or
 - Use the smallest key in the right subtree as the parent of the left subtree.

The height of a tree can be (a) extended or (b) reduced after deleting by merging.

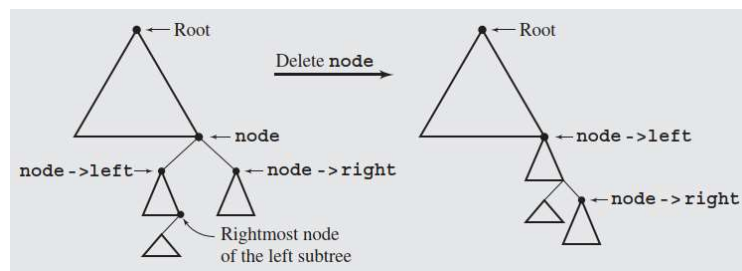


Lecture-12

COMP217 by Dr. Ferhun Yorgancıoğlu

45

How Deletion by Merging Works



Lecture-12

COMP217 by Dr. Ferhun Yorgancıoğlu

46

Deletion by Copying

- If a node has two children, simplify deletion by replacing its key with its immediate predecessor (rightmost node in the left subtree) or successor (leftmost node in the right subtree). This reduces the problem to deleting a leaf or a single-child node.

